

MoTe2 Phase-Change Memristor

Device Model, Verilog-A Implementation & Logic-Gate Simulation Results

1. Python I-V Mathematical Model

```
#DOI: 10.1039/d0nr08725d
import numpy as np
import matplotlib.pyplot as plt

# --- 1. Physical Constants ---
k_B = 8.617e-5          # Boltzmann constant (eV/K)
T = 300                 # Temperature (K)
G0 = 0.4e-5             # DP1 conductance
L = 0.7e-9              # Device thickness (0.7nm)

# --- 2. Model Parameters ---
Ea = 2.0                # Activation Energy (eV)
beta = 0.59e-9          # Field coupling constant (eV*m/V)
k_attempt = 1e10         # Attempt frequency (Hz)

# Schottky Parameters
Is = 1.0e-10
n_set = 29
n_reset = 21
vt = 0.026

# Metallic Parameters
N_channels = 1
G_metallic = N_channels * G0

freq = 1e3              # 1 kHz
t = np.linspace(0, 1/freq, 25000) # 1 cycle
V_max = 4.0             # 4.5V Amplitude
V_signal = V_max * np.sin(2 * np.pi * freq * t)
#V_signal[V_signal < 0] *= 0.4
dt = t[1] - t[0]

class MoTe2:
    def __init__(self):
        # Thresholds
        self.v_off = 0.2
        self.v_on = -0.2
        self.Eset = 3.7
        self.Ereset = 3.3

        # State limits
        self.w_init = 0 # Starts in OFF State
        self.w = self.w_init
        self.p_set = 2
        self.p_reset = 0.8

    def dw_dt(self, v):
        if v > self.v_off:
            window = (1 - self.w)**self.p_set
            return k_attempt * np.exp(-(self.Eset - beta * (v/L)) / (k_B * T))) * window
        elif v < self.v_on:
            # Using -(self.w**p) to ensure a negative rate for RESET
            window = -(self.w)**self.p_reset
            return k_attempt * np.exp(-(self.Ereset - beta * (abs(v/L))) / (k_B * T))) * window
        else:
            return 0
```

```

# --- 3. Simulation Loop ---
memristor = MoTe2()
current = []
state = []
derivative = []

for v in V_signal:
    dw_val = memristor.dw_dt(v)
    derivative.append(dw_val)

    # Update and clip state variable
    memristor.w = np.clip(memristor.w + dw_val * dt, 0, 1)
    state.append(memristor.w)

    # Select ideality factor based on polarity
    if v >= 0:
        n_current = n_set
    else:
        n_current = n_reset

    # Calculate Schottky and Metallic components
    I_sch = Is * (np.exp(np.abs(v) / (n_current * vt)) - 1) * np.sign(v)
    I_met = G_metallic * v

    # Total current blending (HRS vs LRS)
    I_total = (1 - memristor.w) * I_sch + memristor.w * I_met
    I_clipped = np.maximum(np.abs(I_total), 1e-13)
    current.append(I_clipped)

# --- 4. Plotting ---

# Plot 1: State Variable vs Voltage
plt.figure(figsize=(8, 4))
plt.plot(V_signal, state)
plt.title("State Variable (w) vs Voltage")
plt.xlabel("Voltage (V)")
plt.ylabel("State (w)")
plt.grid(True)
plt.show()

# Plot 2: I-V Characteristics (Log Scale)
plt.figure(figsize=(10, 6))
plt.semilogy(V_signal, current, label="Simulated Model", linewidth=2)

# Experimental Data Reference
x_exp = [0.05913, 0.05913, 0.05913, 0.05913, 0.05913, 0.05913, 0.05913, 0.05913,
0.11386, 0.11386, 0.14123, 0.14123, 0.19386, 0.4107, 0.817, 1.16857,
1.57488, 2.06118, 2.49485, 2.90116, 3.30746, 3.65903, 3.82113, 3.82113,
3.82113, 3.82113, 3.82113, 3.82113, 4.03797, 4.03797,
3.87587, 3.63166, 3.38956, 3.01063, 2.71169, 2.44012, 2.14328, 1.81908,
1.52014, 1.1412, 0.84437, 0.54543, 0.30333, 0.14123, 0.03176, 0.03176,
0.00439, 0.00439, -0.02298, -0.04824, -0.10297, -0.18508, -0.29244, -0.53665,
-0.94295, -1.51136, -2.10713, -2.79553, -3.09447, -3.41867, -3.71761, -4.01445,
-4.09655, -4.01445, -3.74287, -3.41867, -3.41867, -3.41867, -3.41867, -3.41867,
-3.41867, -3.41867, -3.41867, -3.2292, -2.93237, -2.4987, -1.83556, -1.37452,
-0.82085, -0.44191, -0.14297, 0.01913]

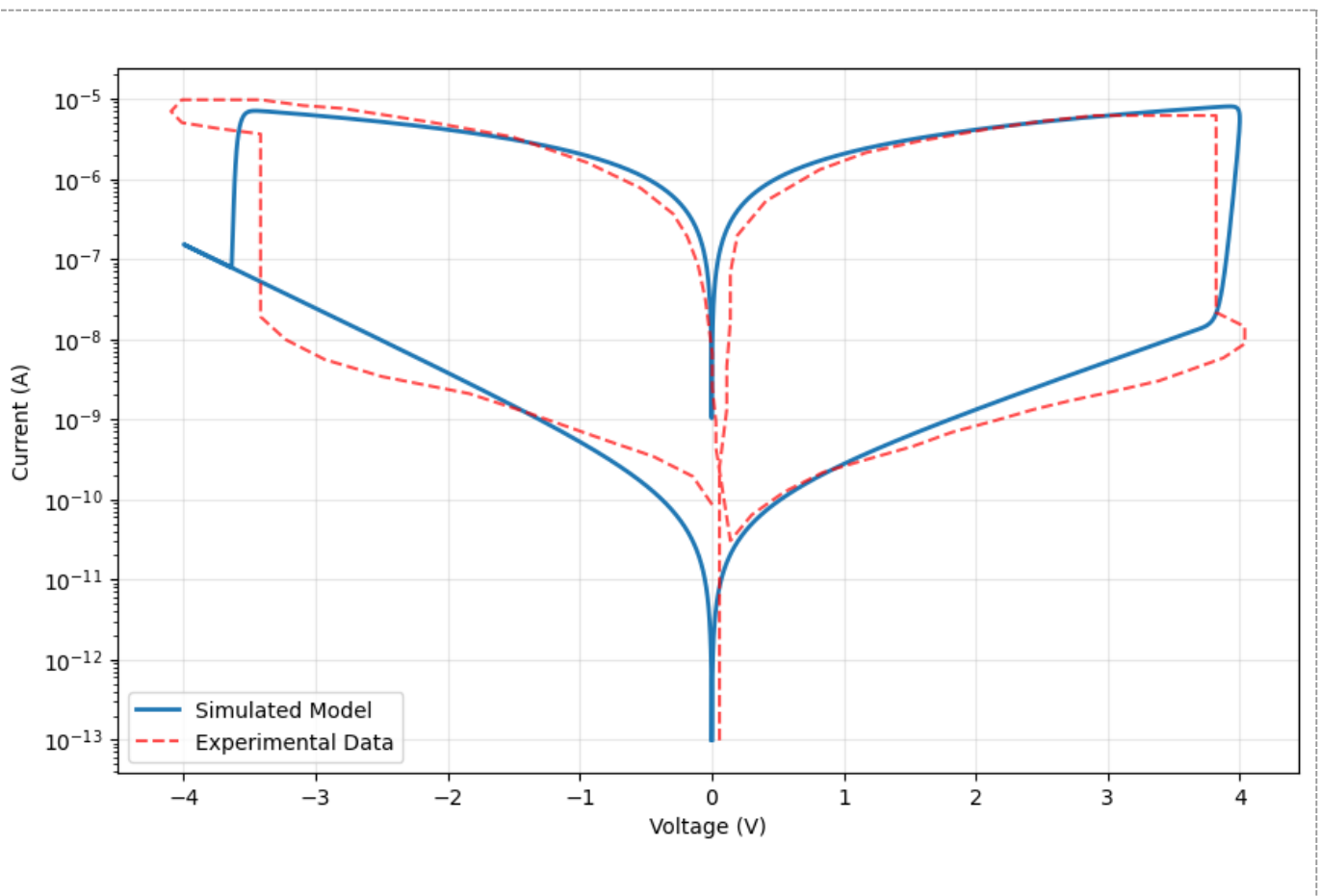
y_exp = [9.83428E-14, 0.0000000000206089, 0.0000000000506541, 0.0000000000159662,
0.0000000000463213, 0.000000000124486, 0.00000000042359, 0.000000000257535,
0.0000000013265, 0.0000000045426, 0.0000000169009, 0.000000067882, 0.000000196939,
0.000000529263, 0.00000130086, 0.00000212577, 0.00000296178, 0.00000410033,
0.00000522488, 0.0000061673, 0.0000061673, 0.0000061673, 0.0000061673, 0.0000044548,
0.0000015355, 0.00000048715, 0.000000196939, 0.0000000801258, 0.0000000351926,
0.0000000215361, 0.0000000143183, 0.00000000876208, 0.00000000582548, 0.00000000418114,
0.00000000302015, 0.00000000216767, 0.00000000170112, 0.0000000013265, 0.000000000958168,

```

```
0.0000000068771, 0.00000000457225, 0.00000000303986, 0.00000000218181, 0.00000000122892,
0.000000000641197, 0.00000000030597, 0.000000000440058, 0.00000000091633, 0.00000000246258,
0.00000000657598, 0.000000148769, 0.0000000311762, 0.0000000832515, 0.000000189545,
0.000000365608, 0.000000761303, 0.0000015954, 0.00000334335, 0.00000502871, 0.00000756367,
0.00000821754, 0.00000969973, 0.00000969973, 0.00000969973, 0.00000696183, 0.00000502871,
0.00000426029, 0.00000363237, 0.00000204596, 0.00000089862, 0.000000397214, 0.000000160581,
0.000000709813, 0.000000338713, 0.000000190782, 0.000000098909, 0.0000000557111,
0.0000000340924, 0.0000000208628, 0.0000000117511, 0.00000000560748, 0.00000000343149,
0.00000000193281, 0.000000000786374]
```

```
plt.semilogy(x_exp, y_exp, 'r--', label="Experimental Data", alpha=0.7)

plt.xlabel("Voltage (V)")
plt.ylabel("Current (A)")
plt.legend()
plt.grid(True, which="major", ls="--", alpha=0.3)
#plt.show()
plt.savefig("WSe2.png", dpi=300)
```



[Python I-V model simulation]

2. Verilog-A Memristor Model (A_Mem_Code_Alpha.va)

```
//  
// MoTe2 Phase Change Memristor  
//  
`include "constants.h"  
`include "discipline.h"  
  
module A_Mem_Code_Alpha(p, n, w_position);  
    input p, n;  
    output w_position;  
    electrical p, n, w_position;  
  
    // --- 1. Universal Physical Constants ---  
    parameter real kB = 8.617e-5;           // Boltzmann constant (eV/K)  
    parameter real T = 300.0;              // Temperature (K)  
    parameter real vt = 0.026;             // Thermal voltage  
    parameter real q_charge = 1.602e-19;    // Elementary electron charge (Coulombs)  
  
    // --- 2. Device Parameters (Table 2, D11: Ag/WSe2) ---  
    parameter real alpha = 5.25e18;         // Barrier Lowering coefficient  
    parameter real k_attempt = 1e10;        // Attempt frequency (Hz)  
  
    // --- 3. Thresholds & Energies ---  
    parameter real v_off = 0.2;             // SET threshold  
    parameter real v_on = -0.2;            // RESET threshold  
    parameter real Eset = 3.7;              // Activation Energy SET (eV)  
    parameter real Ereset = 3.3;            // Activation Energy RESET (eV)  
  
    // --- 4. Conduction Parameters ---  
    parameter real Is = 1.0e-10;           // Schottky Saturation Current  
    parameter real n_set = 29.0;           // Ideality factor (SET)  
    parameter real n_reset = 21.0;         // Ideality factor (RESET)  
    parameter real G_metallic = 0.4e-5;    // Metallic conductance  
  
    // --- 5. State Limits & Exponents ---  
    parameter real w_init = 0.0;           // Starts in OFF State (Semiconducting)  
    parameter real p_set = 2.0;            // Polynomial window for SET  
    parameter real p_reset = 0.8;          // Polynomial window for RESET  
  
    // Internal Variables  
    real w, dw_dt, window;  
    real I_sch, I_met, I_total;  
    real v_val, n_current, exp_arg;  
  
    analog begin  
        v_val = V(p, n);  
  
        // =====  
        // PART 1: The Pure Theoretical Arrhenius Engine  
        // =====  
        if (v_val > v_off) begin  
            // SET Process (Using  $\alpha * q * V$ )  
            exp_arg = -(Eset - (alpha * q_charge * v_val)) / (kB * T);  
            dw_dt = k_attempt * exp(exp_arg) * pow(abs(1.0 - w), p_set);  
  
        end else if (v_val < v_on) begin  
            // RESET Process (Using  $\alpha * q * |V|$ )  
            exp_arg = -(Ereset - (alpha * q_charge * abs(v_val))) / (kB * T);  
            dw_dt = -k_attempt * exp(exp_arg) * pow(abs(w), p_reset);  
  
        end else begin  
            // Inside the dead-zone  
            dw_dt = 0.0;  
        end  
    end  
end
```

```

// =====
// PART 2: SmartSpice Anti-Windup Engine
// =====
window = 1.0;
if ((w >= 0.999) && (dw_dt > 0.0)) window = 0.0;
if ((w <= 0.001) && (dw_dt < 0.0)) window = 0.0;

w = idt(dw_dt * window, w_init);

if (w > 1.0) w = 1.0;
if (w < 0.0) w = 0.0;

// =====
// PART 3: Current Blending Output
// =====
if (v_val >= 0.0) begin
    n_current = n_set;
end else begin
    n_current = n_reset;
end

if (v_val >= 0.0) begin
    I_sch = Is * (exp(v_val / (n_current * vt)) - 1.0);
end else begin
    I_sch = -Is * (exp(abs(v_val) / (n_current * vt)) - 1.0);
end

I_met = G_metallic * v_val;
I_total = (1.0 - w) * I_sch + w * I_met;

V(w_position) <+ w;
I(p, n) <+ I_total;
end
endmodule

```

2.1. Testbench Netlist

```

* Testbench for MoTe2 Phase Change Memristor (Alpha Parameter Model)

* Load the new alpha-based Verilog-A module
.verilog "A_Mem_Code_Alpha.va"

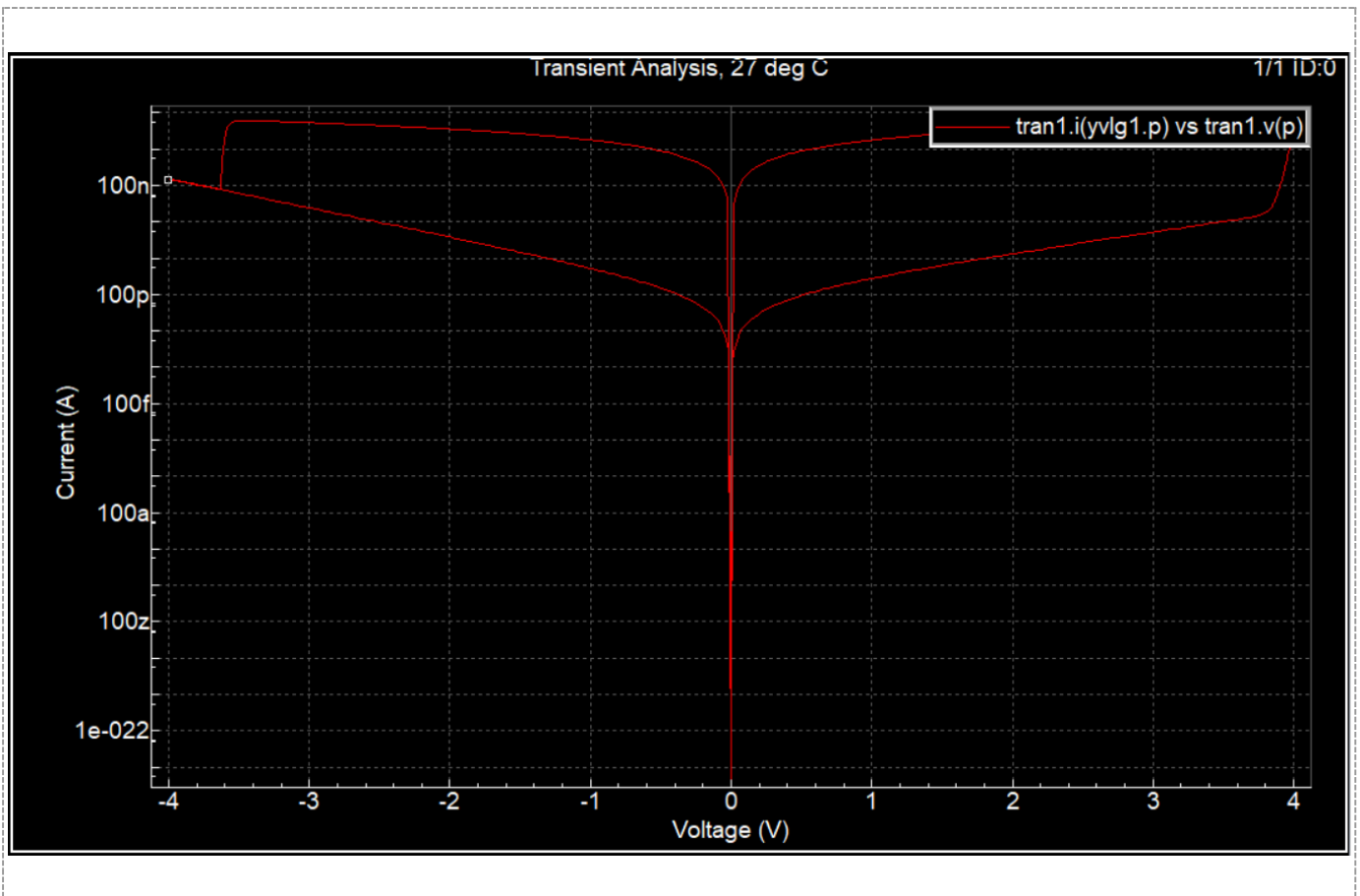
* Drive with a 4.0V sine wave at 1 kHz (matches the Python script)
V1 p 0 SIN(0 4.0 1000)

* Instantiate the Phase Change device
YVLG1 p 0 w_monitor A_Mem_Code_Alpha
Rmon w_monitor 0 1G

* Analysis Setup
* Run for exactly 1 cycle (1 millisecond).
* Using a tiny 100ns step to capture the violent Arrhenius exponential snaps perfectly.
.tran 100n 1m

* Export waveforms for SmartView
.plot v(p) i(YVLG1.p)
.end

```



[Verilog A memristor model simulation]

3. Superimposed Python + Verilog-A Comparison

```
# =====
# PART 1: Physical Parameters & Mathematical Model
# =====
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from google.colab import files

k_B, T, G0, L = 8.617e-5, 300, 0.4e-5, 0.7e-9
beta, k_attempt = 0.59e-9, 1e10
Is, n_set, n_reset, vt = 1.0e-10, 29, 21, 0.026
G_metallic = 0.4e-5
freq, V_max = 1e3, 4.0
t = np.linspace(0, 1/freq, 25000)
V_signal = V_max * np.sin(2 * np.pi * freq * t)
dt = t[1] - t[0]

class MoTe2:
    def __init__(self):
        self.v_off, self.v_on = 0.2, -0.2
        self.Eset, self.Ereset = 3.7, 3.3
        self.w = 0
        self.p_set, self.p_reset = 2.0, 0.8
    def dw_dt(self, v):
        if v > self.v_off:
```

```

        return k_attempt * np.exp((- (self.Eset - beta * (v/L)) / (k_B * T))) * (1 -
self.w)**self.p_set
    elif v < self.v_on:
        return k_attempt * np.exp((- (self.Ereset - beta * (abs(v/L))) / (k_B * T))) * -
(self.w**self.p_reset)
    return 0

memristor = MoTe2()
current = []
for v in V_signal:
    dw_val = memristor.dw_dt(v)
    memristor.w = np.clip(memristor.w + dw_val * dt, 0, 1)
    n_curr = n_set if v >= 0 else n_reset
    I_sch = Is * (np.exp(np.abs(v) / (n_curr * vt)) - 1) * np.sign(v)
    I_met = G_metallic * v
    current.append(np.maximum(np.abs((1 - memristor.w) * I_sch + memristor.w * I_met), 1e-13))

# =====
# PART 2: Base Plotting & Reference Data
# =====
plt.figure(figsize=(10, 6))

# Styled to match exactly: thicker blue line
plt.semilogy(V_signal, current, Label="Python Mathematical Model", color='tab:blue', linewidth=2.5)

x_exp = [0.05913, 0.11386, 0.14123, 0.19386, 0.4107, 0.817, 1.16857, 1.57488, 2.06118, 2.49485,
2.90116, 3.30746, 3.65903, 3.82113, 4.03797, 3.87587, 3.63166, 3.38956, 3.01063, 2.71169, 2.44012,
2.14328, 1.81908, 1.52014, 1.1412, 0.84437, 0.54543, 0.30333, 0.14123, 0.03176, 0.00439, -0.02298, -
0.04824, -0.10297, -0.18508, -0.29244, -0.53665, -0.94295, -1.51136, -2.10713, -2.79553, -3.09447, -
3.41867, -3.71761, -4.01445, -4.09655, -4.01445, -3.74287, -3.41867, -3.2292, -2.93237, -2.4987, -
1.83556, -1.37452, -0.82085, -0.44191, -0.14297, 0.01913]
y_exp = [9.83e-14, 1.32e-9, 1.69e-8, 1.96e-7, 5.29e-7, 1.30e-6, 2.12e-6, 2.96e-6, 4.10e-6, 5.22e-6,
6.16e-6, 6.16e-6, 6.16e-6, 6.16e-6, 1.43e-8, 5.82e-9, 4.18e-9, 3.02e-9, 2.16e-9, 1.70e-9, 1.32e-9,
9.58e-10, 6.87e-10, 4.57e-10, 3.03e-10, 2.18e-10, 1.22e-10, 6.41e-11, 3.05e-11, 4.40e-10, 2.46e-9,
1.48e-8, 3.11e-8, 8.32e-8, 1.89e-7, 3.65e-7, 7.61e-7, 1.59e-6, 3.34e-6, 5.02e-6, 7.56e-6, 8.21e-6,
9.69e-6, 9.69e-6, 9.69e-6, 6.96e-6, 5.02e-6, 4.26e-6, 3.63e-6, 2.04e-6, 8.98e-7, 3.97e-7, 1.60e-7,
7.09e-8, 3.38e-8, 1.90e-8, 9.89e-9, 5.57e-9]

# Styled to match exactly: dashed red line
plt.semilogy(x_exp, y_exp, 'r--', Label="Experimental Data", alpha=0.6, linewidth=1.5)

# =====
# PART 3: Upload and Process Verilog-A Data
# =====
print("Upload your 'verilog_data.txt' (Current:Col 0, Voltage:Col 1)")
uploaded = files.upload()

def parse_spice_safe(val):
    val = str(val).lower().strip()
    m = {'z': 1e-21, 'a': 1e-18, 'f': 1e-15, 'p': 1e-12, 'n': 1e-9, 'u': 1e-6, 'm': 1e-3, 'k': 1e3,
'meg': 1e6}
    try: return float(val)
    except ValueError:
        try: return float(val[:-1]) * m[val[-1]] if val[-1] in m else np.nan
        except: return np.nan

if uploaded:
    fname = list(uploaded.keys())[0]
    df = pd.read_csv(fname, sep=r'\s+', header=0, engine='python')

    # Mapping based on your specific image
    i_raw = df.iloc[:, 0].apply(parse_spice_safe)
    v_raw = df.iloc[:, 1].apply(parse_spice_safe)

    mask = v_raw.notnull() & i_raw.notnull()

```

```

# Using the full array directly since the data is now strictly one cycle
v_clean = v_raw[mask].values
i_clean = np.abs(i_raw[mask].values)

# =====
# PART 4: Final Plotting & Visualization
# =====
plt.semilogy(v_clean, i_clean, color='forestgreen', linestyle='-', label="Verilog-A (Steady-
State)", linewidth=2.5)

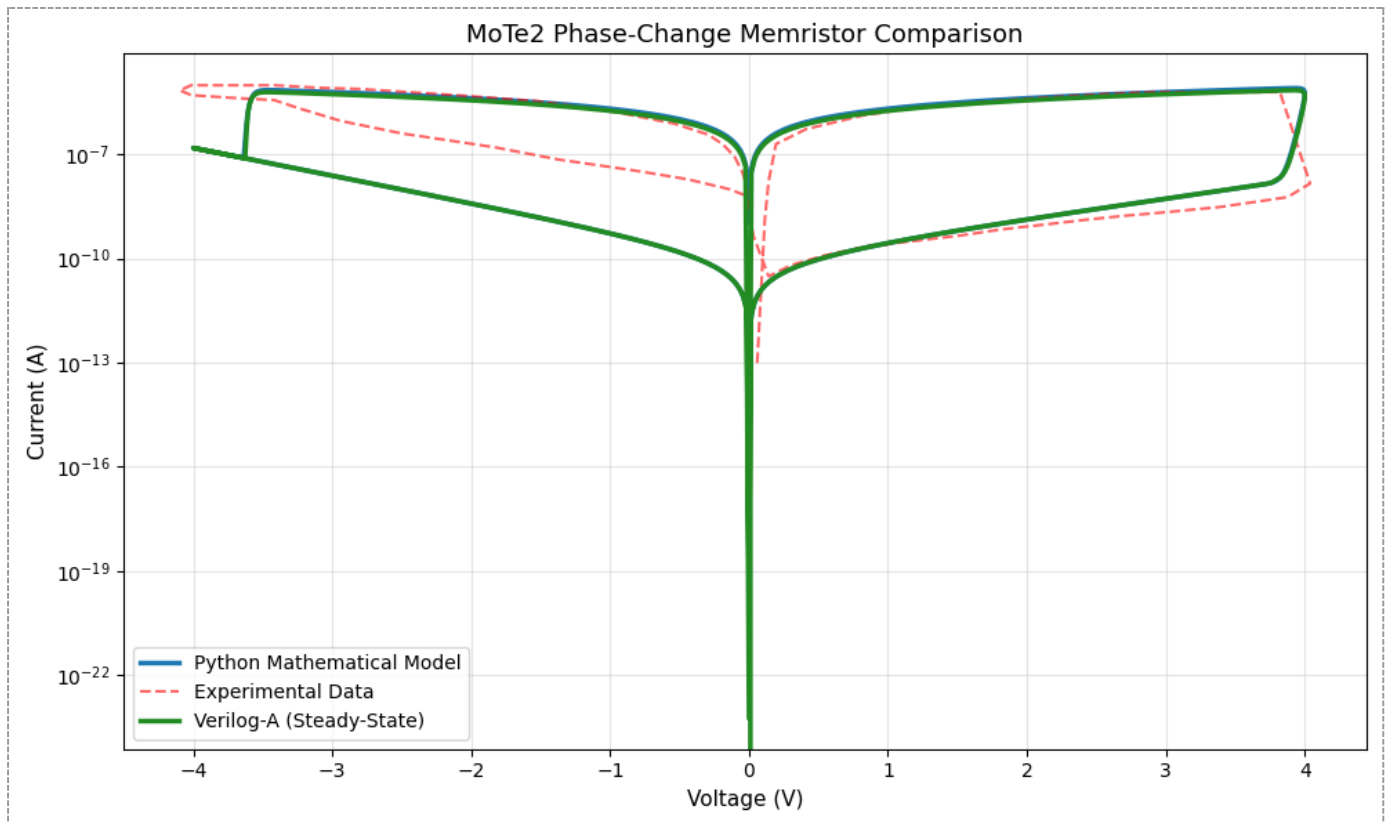
plt.xlabel("Voltage (V)", fontsize=11)
plt.ylabel("Current (A)", fontsize=11)
plt.title("MoTe2 Phase-Change Memristor Comparison", fontsize=13)

# Place Legend exactly in the Lower Left
plt.legend(loc="lower left", fontsize=10)

# Match grid styling
plt.grid(True, which="major", color='lightgray', alpha=0.6)

plt.tight_layout()
plt.show()

```



[Superimposed Python + Verilog-A comparison simulation]

4. Logic-Gate Simulation Results

4.1. NOT Gate

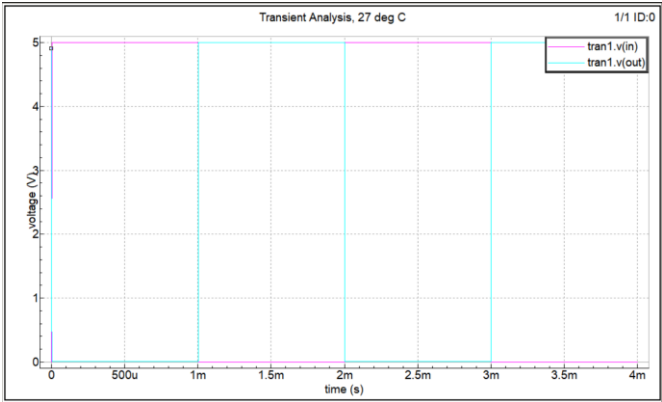
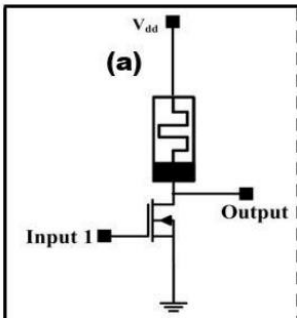
```
* 1. NOT Gate: Pure Memristor Logic
.verilog "A_Mem_Code_Alpha.va"
.model nmos180 nmos Level=1
.options itl4=100 reltol=0.005

Vdd vdd 0 DC 5
Vin1 in1 0 PULSE(0 5 0 1u 1u 2m 4m)

* Dummy source to measure memristor current for I-V curve
Vmeas_mem vdd top DC 0
x1 top out w1 A_Mem_Code_Alpha
r1 w1 0 1G
m1 out in1 0 0 nmos180 L=180n W=10u

.tran 10u 6m
.meas tran avg_power AVG p(Vdd)

* Save all nodes for V_in vs V_out and I-V plotting
.save v(in1) v(out) v(top) i(Vmeas_mem)
.plot tran v(in1) v(out)
.end
```



Resistor-Based Power	Memristor-Based Power (Avg.)
125.0 μ W	66.65 μ W

4.2. NAND Gate

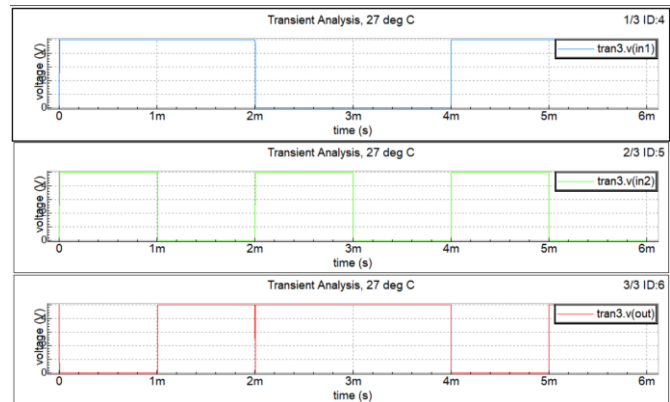
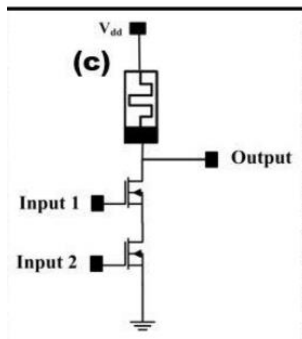
```
* 2. NAND Gate: Pure Memristor Logic
.verilog "A_Mem_Code_Alpha.va"
.model nmos180 nmos Level=1
.options itl4=100 reltol=0.005

Vdd vdd 0 DC 5
Vin1 in1 0 PULSE(0 5 0 1u 1u 2m 4m)
Vin2 in2 0 PULSE(0 5 0 1u 1u 1m 2m)

* Dummy source to measure memristor current for I-V curve
Vmeas_mem vdd top DC 0
x1 top out w1 A_Mem_Code_Alpha
r1 w1 0 1G
m1 out in1 mid 0 nmos180 L=180n W=10u
m2 mid in2 0 0 nmos180 L=180n W=10u
```

```
.tran 10u 6m
.meas tran avg_power AVG p(Vdd)

.save v(in1) v(in2) v(out) v(top) i(Vmeas_mem)
.plot tran v(in1) v(in2) v(out)
.end
```



Resistor-Based Power	Memristor-Based Power (Avg.)
83.3 μ W	33.38 μ W

4.3. AND Gate

```
* 4. AND Gate: Cascaded Memristor Logic (Fixed Math)
.verilog "A_Mem_Code_Alpha.va"
.model nmos180 nmos Level=1
.options itl4=1000 reltol=0.005

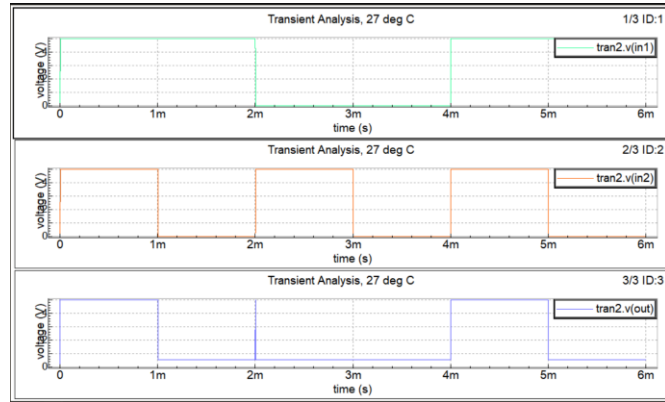
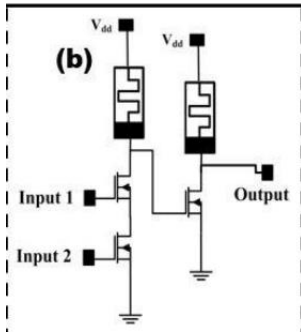
Vdd vdd 0 DC 5
Vin1 in1 0 PULSE(0 5 0 1u 1u 2m 4m)
Vin2 in2 0 PULSE(0 5 0 1u 1u 1m 2m)

* --- STAGE 1: NAND Gate ---
Vmeas_mem1 vdd top1 DC 0
x1 top1 n_out w1 A_Mem_Code_Alpha
r1 w1 0 1G
m1 n_out in1 mid 0 nmos180 L=180n W=10u
m2 mid in2 0 0 nmos180 L=180n W=10u

* --- STAGE 2: NOT Gate ---
Vmeas_mem2 vdd top2 DC 0
x2 top2 out w2 A_Mem_Code_Alpha
r2 w2 0 1G
m3 out n_out 0 0 nmos180 L=180n W=10u

.tran 10u 6m
.meas tran avg_power AVG p(Vdd)

.save v(in1) v(in2) v(n_out) v(out) v(top1) v(top2) i(Vmeas_mem1) i(Vmeas_mem2)
.plot tran v(in1) v(in2) v(out)
.end
```



Resistor-Based Power	Memristor-Based Power (Avg.)
249.4 μ W	99.91 μ W

4.4. NOR Gate

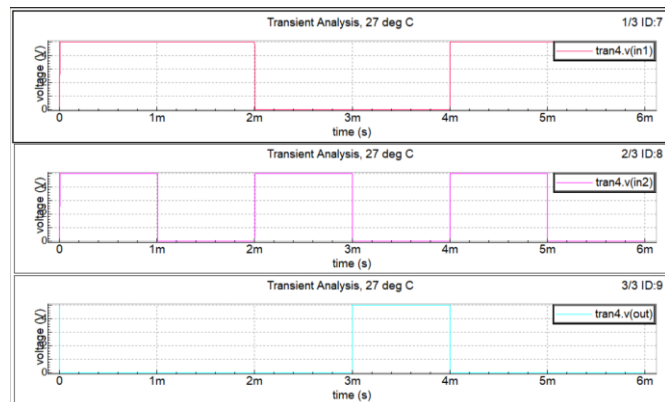
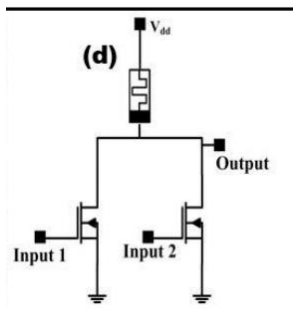
```
* 3. NOR Gate: Pure Memristor Logic
.verilog "A_Mem_Code_Alpha.va"
.model nmos180 nmos level=1
.options itl4=100 reltol=0.005
```

```
Vdd vdd 0 DC 5
Vin1 in1 0 PULSE(0 5 0 1u 1u 2m 4m)
Vin2 in2 0 PULSE(0 5 0 1u 1u 1m 2m)
```

```
Vmeas_mem vdd top DC 0
x1 top out w1 A_Mem_Code_Alpha
r1 w1 0 1G
m1 out in1 0 0 nmos180 L=180n W=10u
m2 out in2 0 0 nmos180 L=180n W=10u
```

```
.tran 10u 6m
.meas tran avg_power AVG p(Vdd)
```

```
.save v(in1) v(in2) v(out) v(top) i(Vmeas_mem)
.plot tran v(in1) v(in2) v(out)
.end
```



Resistor-Based Power	Memristor-Based Power (Avg.)
208.1 μ W	83.32 μ W

4.5. OR Gate

```
* 5. OR Gate: Cascaded Memristor Logic (Fixed Math)
.verilog "A_Mem_Code_Alpha.va"
.model nmos180 nmos Level=1
.options itl4=100 reltol=0.005

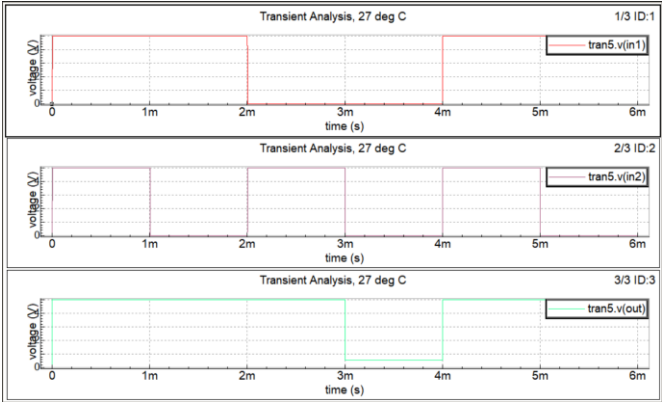
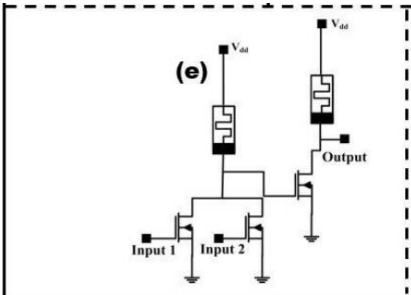
Vdd vdd 0 DC 5
Vin1 in1 0 PULSE(0 5 0 1u 1u 2m 4m)
Vin2 in2 0 PULSE(0 5 0 1u 1u 1m 2m)

* --- STAGE 1: NOR Gate ---
Vmeas_mem1 vdd top1 DC 0
x1 top1 nor_out w1 A_Mem_Code_Alpha
r1 w1 0 1G
m1 nor_out in1 0 0 nmos180 L=180n W=10u
m2 nor_out in2 0 0 nmos180 L=180n W=10u

* --- STAGE 2: NOT Gate ---
Vmeas_mem2 vdd top2 DC 0
x2 top2 out w2 A_Mem_Code_Alpha
r2 w2 0 1G
m3 out nor_out 0 0 nmos180 L=180n W=10u

.tran 10u 6m
.meas tran avg_power AVG p(Vdd)

.save v(in1) v(in2) v(nor_out) v(out) v(top1) v(top2) i(Vmeas_mem1) i(Vmeas_mem2)
.plot tran v(in1) v(in2) v(out)
.end
```



Resistor-Based Power	Memristor-Based Power (Avg.)
249.6 μ W	99.94 μ W

6. Power Consumption Summary

